

ORION VLSI TECHNOLOGIES

Final Project

BMU Verification Plan



RAWABI VLSI DESIGN CENTER

ORION
VLSI Technologies

PROVIDING SERVICES FOR
THE COMPLETE VLSI DESIGN CYCLE

Contents

List Of Abbreviations	3
Introduction	4
Goals of This Document	4
Scope of Verification	4
Assumptions and Constraints	4
Constraints	4
Assumptions	4
Design Overview	5
Verification Strategy	6
Testbench Structure	11
Verification Extra Details	12
Tools and Technologies	12
Regression Execution	12
Known DUT Findings, Results, and Sign-Off	13
Confirmed DUT Findings	13
Coverage Reports	13
Functional Coverage Results	13
Code Coverage Results	14
Verification Results	14
Risk Management	14
Sign-Off Criteria	15
Open Items, Assumptions and Clarification Requests	16

List Of Abbreviations

BMU	Bit Manipulation Unit
UVM	Universal Verification Methodology
DUT	Design Under Test
RTL	Register Transfer Level
OP	Operation
RISC-V	RISC-V Instruction Set Architecture
Zbb	Basic Bit-Manipulation Subset
Zba	Address-Generation Bit-Manipulation Subset
Zbs	Single-Bit Manipulation Subset
Zbp	Permutation Bit-Manipulation Subset
SRL / SRA	Shift Right Logical / Arithmetic
SLL / ROL / ROR	Shift Left Logical / Rotate Left / Rotate Right
CLZ / CTZ	Count Leading Zeros / Count Trailing Zeros
CPOP	Count Population (set-bit count)
SLT / SLTU	Set-on-Less-Than (signed / unsigned)
MSB / LSB	Most / Least Significant Bit
CSR	Control and Status Register
VP	Verification Plan
TB	Testbench
DV	Design Verification
ISA	Instruction Set Architecture
ap	Decoded instruction control-signal struct (rtl_alu_pkt_t)

Introduction

Goals of This Document

This document defines the verification plan for the Bit Manipulation Unit (BMU), a synthesizable RTL block intended to implement selected operations associated with the RISC-V Bit Manipulation extensions. It records the approved scope, timing rules, testbench architecture, checking strategy, planned stimulus, coverage model, measured results, known DUT findings, and remaining sign-off items. The document serves as the technical reference for reviewing and reproducing the BMU UVM verification effort.

Scope of Verification

- The verification scope includes the top-level Bit_Manipulation_Unit and all associated logic contained within its design boundary.
- The BMU is verified as a standalone block through its documented top-level input and output interface.
- The functional scope includes the following 18 operations: OR, ORN, XOR, XNOR, SRL, SRA, ROR, BINV, SH2ADD, SUB, SLT, SLTU, CTZ, CPOP, SEXT.B, Signed MAX, PACK, and GREV/REV8.
- The scope also includes the documented CSR read and write functionality.
- The common BMU interface behavior associated with ap, operands, valid_in, reset, result_ff, and error is included.
- Verification is limited to normal functional operation using the approved parameter configuration with scan_mode = 0.

Assumptions and Constraints

Constraints

- The DUT RTL shall not be modified during the verification process.
- The BMU shall be verified as a standalone, black-box design.
- Verification shall be limited to the 18 approved BMU operations and the three documented CSR modes.
- scan_mode shall remain deasserted during functional verification.
- Undefined or insufficiently documented behavior shall not be assigned an expected result until clarified.

Assumptions

- The design and verification environment use UVM — confirmed by the supplied architecture skeleton.
- The BMU specification, together with the approved design clarifications, provides the authoritative definition of the expected DUT behavior.
- The supplied RTL files, package definitions, parameters, and interface types represent the intended and mutually compatible project revision.
- The required simulation environment, UVM library, clock, and reset stimulus are available and correctly connected to the DUT.

Design Overview

The Bit Manipulation Unit (BMU) is a synthesizable, 32-bit RTL execution block intended to implement arithmetic, logical, shifting, counting, comparison, sign-extension, packing, and bit-manipulation functions associated with the RISC-V Bit Manipulation extensions.

The BMU receives two 32-bit operands and a decoded control packet, `ap`, rather than a complete instruction encoding. Based on these inputs, it selects the requested operation, generates the corresponding result, and detects invalid or conflicting control combinations. The block also supports CSR read and write data-selection paths.

The computed result is captured in the registered output `result_ff`. For a request accepted with `valid_in = 1` in cycle `N`, the corresponding result is available in cycle `N+1`. When `valid_in = 0`, `result_ff` retains its previous value.

Key Features

- Implements selected operations associated with the RISC-V Bit Manipulation extensions listed below.
- Integrated control and data path.
- Provides compile-time feature enablement through the `pt.BITMANIP_*` parameter set.
- Control-legality error output for the documented invalid and conflicting control combinations.
- Produces a registered 32-bit result with one-cycle latency.

Supported Extensions

The following table lists the operations identified in the specification's Features section. This list describes the advertised design capabilities and does not define the current verification scope. The operations selected for verification are identified separately in the Scope of Verification section.

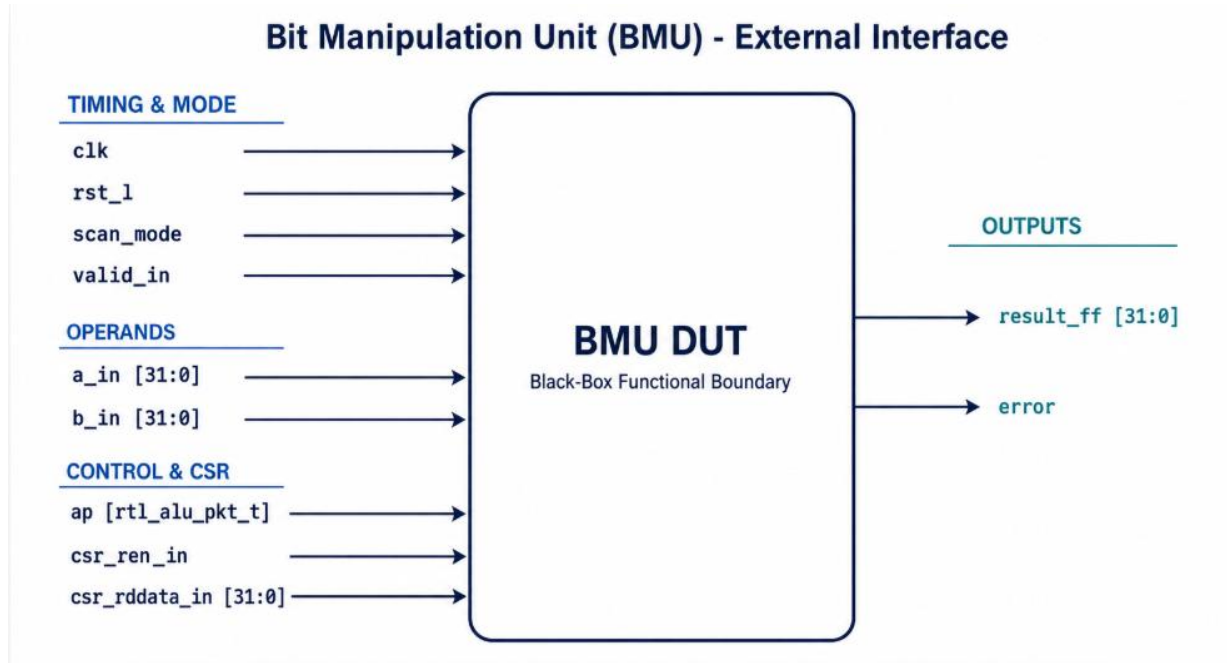
Extension	Operations (as named in the specification)
Zbb	CLZ, CTZ, CPOP, MIN, MAX, ANDN, ORN, XNOR, SEXT.B, SEXT.H
Zbs	BSET, BCLR, BINV, BEXT
Zbp	ROL, ROR, PACK, PACKU, PACKH, GREV (rev8 subset), GORC (orc b subset)
Zba	SH1ADD, SH2ADD, SH3ADD

Each instruction is conditionally enabled based on its corresponding `pt.BITMANIP_XXX` compile-time parameter, which is distinct from the runtime `ap.zbb`-style qualifier bits used within individual operations.

Block Diagram and Interface

The interface and signal details are listed below.

Signal	Width	Direction	Description
<code>clk</code>	1 bit	Input	Clock signal to the module
<code>rst_l</code>	1 bit	Input	Active-low synchronous reset; resets all internal registers to their initial state
<code>a_in</code>	32 bits	Input	First operand of the BMU module
<code>b_in</code>	32 bits	Input	Second operand of the BMU module
<code>scan_mode</code>	1 bit	Input	Scan test-mode control, held at 0 during functional verification
<code>valid_in</code>	1 bit	Input	Instruction valid flag — indicates whether the input instruction is valid
<code>ap</code>	Struct	Input	Decoded instruction control signals (<code>rtl_alu_pkt_t</code> in the RTL/testbench)
<code>csr_ren_in</code>	1 bit	Input	CSR read-enable
<code>csr_rddata_in</code>	32 bits	Input	CSR read data
<code>result_ff</code>	32 bits	Output	Final computed result
<code>error</code>	1 bit	Output	Error indicator

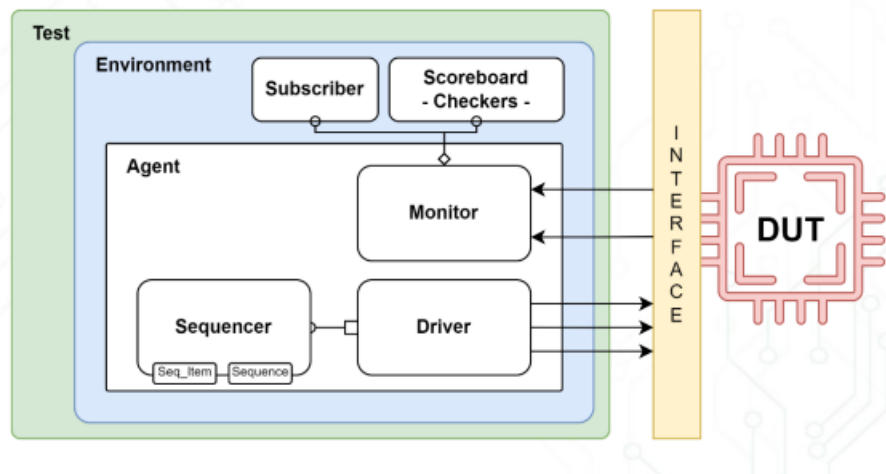


Verification Strategy

Methodology

SystemVerilog and UVM shall be used to verify the BMU as a standalone DUT through directed and constrained-random simulation.

The testbench shall include a sequence item, sequences, sequencer, driver, monitor, agent, scoreboard, functional-coverage collector, environment, tests, interface, and top-level testbench module.



An independent reference model shall be implemented in SystemVerilog and used by the scoreboard to predict the expected BMU results based on the specification and approved design clarifications.

Selection of Verification Tests

The approved verification baseline covers 18 documented BMU operations and three CSR data-selection modes.

The covered functional groups are:

1. **Logical Operations**
 - OR
 - ORN
 - XOR
 - XNOR
2. **Shift, Rotate, and Bit-Index Operations**
 - SRL
 - SRA
 - ROR
 - BINV
3. **Arithmetic and Comparison Operations**
 - SH2ADD
 - SUB
 - SLT
 - SLTU
 - Signed MAX
4. **Counting and Sign-Extension Operations**
 - CTZ
 - CPOP
 - SEXT.B
5. **Packing and Generalized-Reverse Operations**
 - PACK
 - GREV with the documented REV8 mode
6. **CSR Data-Selection Operations**
 - CSR read
 - CSR write using operand A
 - CSR write using operand B

Additional tests cover synchronous reset, valid_in behavior, result retention, legal operation transitions, invalid and conflicting ap combinations, unsupported GREV mode selections, and recovery from an error-producing request.

The companion workbook, BMU_Test_Plan_last_Updated.xlsx, is the primary test-tracking document for this verification effort.

Specific Corner Cases

The verification tests include the following operation-specific and interface-level corner cases:

1. **Logical Operations**
 - Zero and all-one operands.
 - Equal and complementary operands.
 - Alternating-bit patterns.
 - Complete per-bit logical truth tables.
 - Transitions between the base and qualified operation modes.

2. **Shift and Rotate Operations**
 - Shift-amount classes that preserve distinct behavior: 0, 1-15, 16, 17-30, and 31 for SRL, SRA, and ROR. Directed boundary values are supplemented by constrained-random amounts.
 - Positive and negative operand patterns.
 - One-hot and walking-bit operands.
 - Verification that upper unused bits of `b_in` do not affect the result.
3. **Bit Inversion**
 - BINV index boundaries and regions: bit 0, bits 1-15, bits 16-30, and bit 31, together with both possible original values of the selected bit.
 - Original selected-bit values of both zero and one.
 - Verification that exactly one selected bit is inverted.
4. **Arithmetic Operations**
 - Zero operands and equality cases.
 - Carry and borrow propagation.
 - Modulo-2³² wraparound.
 - Maximum positive, minimum negative, and boundary operand values.
 - SH2ADD behavior when significant bits are shifted out.
5. **Comparison Operations**
 - Less-than, equal, and greater-than relations.
 - All operand-sign combinations.
 - Signed and unsigned interpretation boundaries.
 - Cases where signed and unsigned comparisons produce different results.
6. **Counting Operations**
 - CTZ results from 0 through 32.
 - Explicit verification that CTZ(0) returns 32.
 - CPOP semantic count classes: 0, 1, 2-7, 8-15, 16, 17-31, and 32, plus one-hot samples in each byte region.
 - Walking-one, walking-zero, alternating, zero, and all-one patterns.
7. **Sign Extension**
 - SEXT.B boundaries 00, 7F, 80, and FF, plus representative positive and negative interior values.
 - Positive and negative sign-extension cases.
 - Verification that upper input bits do not affect the result.
8. **PACK**
 - Independent propagation of every bit from both source halfwords.
 - Asymmetric operand patterns to detect source-order reversal.
 - Verification that the unused upper halves do not affect the result.
9. **GREV/REV8**
 - The documented legal REV8 mode.
 - One-hot byte-reversal patterns.
 - Unsupported mode-select values.
 - Verification that unsupported mode-select values produce `.result = 0` and `error = 0` when no control conflict exists.
10. **CSR Paths**
 - CSR read-data selection.
 - CSR write selection from operand A.
 - CSR write selection from operand B.
 - Zero, all-one, one-hot, and structured data patterns.
 - Reviewed CSR and BMU-control conflicts.
11. **Reset and valid_in**
 - Reset at startup and during active traffic.
 - Reset with `valid_in` equal to both zero and one.
 - Result update one cycle after a request accepted with `valid_in = 1`.
 - Result retention for short and extended `valid_in = 0` intervals.
 - Consecutive accepted operations without idle cycles.
 - Verification that synchronous reset clears `result_ff` to 0.

- Verification that synchronous reset clears error to 0, including when valid_in = 1.
- 12. Error and Recovery**
- Legal operation following an error-producing request.
 - SH2ADD selected without the required zba qualifier: expected result = 0 and error = 1.
 - SUB selected with the zba qualifier asserted: expected result = 0 and error = 1.
 - An additional forbidden ap control field asserted while OR, XOR, SRL, SRA, ROR, BINV, or GREV is selected: expected result = 0 and error = 1.
 - Concurrent CSR-read and BMU-operation selection: expected result = 0 and error = 1.
 - All five confirmed error-intent classes are covered, and error stimulus is observed with valid_in = 0 and valid_in = 1. A full reason-by-valid Cartesian cross is intentionally not required because valid_in does not change the combinational error rule.
 - Multiple primary operation fields asserted together: expected result = 0 and error = 1 when the combination does not match a documented legal recipe.
 - A documented legal primary recipe with one additional forbidden ap field asserted: expected result = 0 and error = 1. One isolated forbidden field is sufficient to exercise this error class; a full Cartesian cross would add repetition without a distinct requirement.
 -

Coverage Targets

Functional Coverage

The functional-coverage model is derived from the approved verification scope and meaningful behavioral risks. It measures whether required stimulus classes were exercised; it does not decide whether the DUT result is correct. Result, error, reset, hold, and one-cycle latency correctness remain scoreboard responsibilities. Equivalent interior values are grouped when they exercise the same requirement, while true boundaries and distinct control paths remain separate bins.

Coverage Group	Bin Distribution	Why It Is Needed
cg_operation	21 = 18 BMU operations + 3 CSR paths	Proves every supported legal mode was accepted at least once.
cg_logic	16 = 4 operations x 4 bit pairs	Covers OR, ORN, XOR, and XNOR for input pairs 00, 01, 10, and 11. Bit position is not crossed because full 32-bit correctness is checked by the scoreboard.
cg_shift	15 = 3 operations x 5 amount classes	Separates 0, 1-15, 16, 17-30, and 31 for SRL, SRA, and ROR. These retain no-shift, half-width, maximum, and interior behavior without 96 repetitive bins.
cg_sra_sign	2 = positive and negative A	Checks zero-fill versus sign-fill risk for nonzero arithmetic shifts; amount behavior is already covered by cg_shift.
cg_binv	6 = 4 index regions + 2 original-bit values	Keeps real index boundaries 0 and 31, groups interior halves, and proves inversion from both 0 to 1 and 1 to 0. No redundant Cartesian cross is required.
cg_compare	24 reachable cases	Covers SLT, SLTU, and signed MAX across sign pairs and <, =, > relations. Twelve mathematically impossible sign/relation combinations are ignored.
cg_shift_data	12 = 3 operations x 4 A classes	For each SRL, SRA, and ROR, covers A equal to zero, all ones, one-hot, and other structured/random data.
cg_sub	5 arithmetic-risk classes	Separates equality, borrow/no-borrow, and signed-overflow/no-overflow behavior.
cg_sh2add	3 arithmetic-risk classes	Separates normal addition, addition carry, and loss of A upper bits after the left shift.
cg_ctz	33 exact results = 0 through 32	Each count is architecturally distinct: 0-31 identify the first set bit and 32 is the corrected all-zero result.
cg_cpop_count	7 semantic count classes	Covers zero, one, low, mid-low, half, high, and full population counts without treating every interior count as a separate requirement.
cg_cpop_region	4 one-hot byte regions	Proves all four bytes contribute to the 32-bit count and targets the observed partial-width implementation risk.
cg_sext_b	10 = 6 byte classes + 4 sign/upper-A combinations	Covers 00, 7F, 80, FF and positive/negative interiors, then proves both signs with upper A equal to zero and nonzero.
cg_pack	3 low-half classes	Covers both low halves zero, both all ones, and all other asymmetric/distinct cases. Per-bit correctness and lane order are checked by the scoreboard.

cg_grev	2 = REV8 and unsupported	Separates the only supported mode, B[4:0]=24, from the common unsupported-mode behavior.
cg_csr_source	3 distinguishable source paths	Proves read-data, write-A, and write-B were tested with selected data different from the alternative sources, so a mux reversal is observable.
cg_csr_data	9 = 3 paths x 3 data classes	For each CSR path, covers zero, all ones, and other data. Exact data correctness is checked by the scoreboard.
cg_error_reason	5 documented error intents	Covers CSR conflict, SH2ADD without Zba, SUB with Zba, multiple primary operations, and an extra forbidden field.
cg_error_valid	2 = valid_in low and high	Implements the trainer clarification that error detection is independent of valid_in. This is intentionally not crossed with every reason.

Reset, result hold, valid_in transitions, recovery, and one-cycle latency are intentionally checked by directed sequences and the cycle-accurate scoreboard rather than counted again as functional covergroups. This avoids duplicate bins whose only effect would be to inflate the percentage.

Code Coverage

Code coverage is collected by Cadence Xcelium for the elaborated Bit_Manipulation_Unit hierarchy and reviewed in IMC. Official simulator metrics are reported without manually changing the denominator.

- Block Coverage: reports Xcelium-instrumented procedural block points after elaboration.
- Expression Coverage: reports Xcelium-instrumented expression conditions; the denominator may be small and does not represent every source-level expression.
- Toggle Coverage: measures observed bit transitions and is expected to expose inactive, configuration-disabled, or integration-only logic more clearly than block coverage in this standalone setup.
- All uncovered items are reviewed as reachable, unreachable, configuration-disabled, integration-only, or outside the approved scope. No arbitrary percentage alone constitutes sign-off.

Test and Result Tracking

The companion workbook, BMU_Test_Plan_last_Updated.xlsx, is the primary test-tracking document for this verification effort.

It contains:

- Implemented operation, control, reset, valid_in, CSR, smoke, and regression tests tracked by the repository and workbook.
- Operation-specific and infrastructure scenarios are tracked in the companion workbook. The workbook remains the authority for scenario identifiers, execution status, evidence, and issue linkage
- Operation-specific stimulus and expected behavior.
- Coverage IDs linked to every scenario.
- Execution gates and priorities.
- Status, test date, and actual-result fields.
- Evidence, issue, and responsibility notes.
- Functional-coverage and cross-coverage registers.
- Run-log and open-decision tracking.

A scenario may be marked **Passed** only after all mandatory scoreboard comparisons, interface checks, and resolved error checks have completed successfully. Coverage percentages must be entered only from an actual simulator-generated coverage report.

Simulation results are now available from the consolidated bmu_regression_test run. The workbook and verification plan must therefore report measured results rather than planning placeholders. The scoreboard checks result_ff with one-cycle alignment and checks error in the current sampled cycle, consistent with the approved timing clarification.

Testbench Structure

Testbench Components

The BMU verification environment consists of the following UVM components:

- **Sequence Item:** Represents one BMU transaction and contains the DUT input controls, operands, and monitored outputs required for checking and coverage.
- **Sequencer:** Supplies sequence items generated by the active sequence to the driver.
- **Driver:** Converts sequence items into pin-level stimulus and drives the BMU input signals through the virtual interface.
- **Monitor:** Samples the DUT inputs and outputs without driving them, constructs observed transactions, and publishes them to the scoreboard and coverage collector.
- **Agent:** Groups the sequencer, driver, and monitor and connects the sequencer to the driver.
- **Reference Model:** Independently calculates the expected BMU result and expected control behavior for each supported request.
- **Scoreboard:** Uses the reference-model prediction to compare the expected response with the monitored result_ff and error, while accounting for reset, valid_in, and the registered result latency.
- **Coverage Collector:** *Samples monitored transactions and measures legal operations, meaningful operand/data classes, comparison relations, CSR source selection, and documented error stimulus. Reset, hold, valid transitions, recovery, and one-cycle latency are checked by directed sequences and the scoreboard rather than duplicated as percentage-driven covergroups.*
- **Environment:** Instantiates and connects the agent, scoreboard, and coverage collector.
- **Sequences and Tests:** Generate the directed, constrained-random, negative, reset, CSR, timing, and mixed-operation scenarios defined in the test-plan workbook.

Testbench Interfaces and Connections

The SystemVerilog interface `bmu_if` provides the pin-level connection between the UVM testbench and the BMU DUT. It contains the following DUT signals:

- `clk`
- `rst_l`
- `scan_mode`
- `valid_in`
- `a_in`
- `b_in`
- `ap`
- `csr_ren_in`
- `csr_rddata_in`
- `result_ff`
- `error`

The interface defines two clocking blocks:

- `cb_drv`, synchronized to the negative edge of `clk`, is used by the driver to apply DUT inputs.
- `cb_mon`, synchronized to the positive edge of `clk`, is used by the monitor to sample DUT inputs and outputs.

The interface also declares `DRV` and `MON` modports. However, the current driver and monitor obtain a handle of type `virtual bmu_if` and access the corresponding clocking blocks directly.

The top-level module, `bmu_tb_top.sv`, creates the physical interface instance and passes its virtual handle to the agent components through `uvm_config_db`.

Internal UVM communication uses transaction-level connections:

- The sequencer transfers sequence items to the driver through the standard sequencer-driver connection.
- The monitor publishes observed transactions through its analysis port.
- The environment connects the monitor analysis port to the scoreboard and coverage subscriber.

These UVM transaction-level connections are separate from the signal-level `bmu_if` connection between the testbench and DUT.

The driver and monitor access this interface through virtual-interface handles. The driver uses the `cb_drv` clocking block to drive DUT inputs on the negative clock edge, while the monitor uses the `cb_mon` clocking block to sample the

Project Structure

The final repository is organized so verification responsibilities are separated clearly:

Path	Purpose
<code>dut_rm/</code>	Independent BMU reference model.
<code>rtl/</code>	Delivered DUT RTL and parameter/type support files.
<code>tb/env/agent/</code>	Driver, monitor, sequencer, sequence item, and agent.
<code>tb/env/scoreboard/</code>	Cycle-accurate scoreboard and reporting.
<code>tb/env/coverage/</code>	Functional coverage subscriber.
<code>tb/interface/</code>	<code>bmu_if</code> clocking blocks and signal interface.
<code>tb/sequences/</code>	Operation and infrastructure sequences.
<code>tb/tests/</code>	Focused tests, reference-model unit test, and <code>bmu_regression_test</code> .
<code>tb/packages/</code>	<code>bmu_pkg</code> compilation package.

Verification Extra Details

Tools and Technologies

Tool	Intended Use
Cadence Xcelium (xrun)	SystemVerilog/UVM compilation, elaboration, simulation, and coverage generation.
Integrated Metrics Center (IMC)	Functional/code-coverage analysis, detailed reports, and coverage-hole review.
Cadence SimVision	Waveform inspection and interactive simulation/debug through the Makefile wave target.
MobaXterm	Remote access to the Orion training server and graphical forwarding.
Visual Studio Code	Source development, review, and project organization.
Git and GitHub	Version control, backup, change tracking, and final source delivery.

Regression Execution

The final regression is executed through `bmu_regression_test`. The test creates the approved BMU sequences and starts them sequentially on the common sequencer within one UVM simulation. This produces one consolidated scoreboard log and one coverage database for the full regression.

- The regression order begins with reset, then logic, shift/rotate/bit-index, arithmetic/comparison, count/sign-extension, MAX/PACK/GREV, CSR, `valid_in`, and negative/error sequences.
- Individual operation tests remain available for focused reruns and waveform debugging.
- The final Makefile supports:
 - `make run : TEST=<test> SEED=<seed>`,
 - `make regression SEED=<seed>`,
 - `make coverage TEST=<test> SEED=<seed>`,
 - `make imc TEST=<test>`, `make wave TEST=<test> SEED=<seed>`, and `make clean`.
- Coverage databases are stored per selected test; use `make imc TEST=bmu_regression_test` to open the consolidated regression database explicitly.

Known DUT Findings, Results, and Sign-Off

Confirmed DUT Findings

The verification environment has exposed multiple specification-to-RTL mismatches. These are treated as DUT findings rather than testbench failures when the reference model and specification agree.

ID	Area	Finding	Status
BUG-01	CPOP	Full 32-bit population count is not implemented correctly; observed behavior indicates counting is limited to part of the operand.	Confirmed DUT defect
BUG-02	CTZ	Some one-hot operand positions produce an incorrect trailing-zero count.	Confirmed DUT defect
BUG-03	PACK	Source halfword ordering differs from the expected {B[15:0], A[15:0]} behavior.	Confirmed DUT defect
BUG-04	CSR write	Immediate/source selection between operand A and operand B is reversed for the affected CSR write path.	Confirmed DUT defect
BUG-05	GREV/REV8	Legal REV8 mode produces incorrect byte ordering.	Confirmed DUT defect
BUG-06	Control legality	Forbidden/conflicting ap controls are accepted in cases where error = 1 is expected.	Confirmed DUT defect
BUG-07	SLT qualification	Required SUB qualification/guard behavior is missing or inconsistent for affected comparison requests.	Confirmed DUT defect

Coverage Reports

Coverage was collected from the final `bm_u_regression_test` execution using Cadence Xcelium and analyzed in the Integrated Metrics Center (IMC). The regression exercised the complete approved BMU/CSR verification scope within one UVM simulation and generated consolidated functional- and code-coverage evidence.

Functional Coverage Results

Coverage Group	Measured Result
Operation	100.00% (21/21)
Logic	100.00% (16/16)
Shift	100.00% (15/15)
SRA Sign	100.00% (2/2)
BINV	100.00% (6/6)
Compare	100.00% (24/24)
Shift Data	100.00% (12/12)
SUB	100.00% (5/5)
SH2ADD	100.00% (3/3)
CTZ	100.00% (33/33)
CPOP Count	100.00% (7/7)
CPOP Region	100.00% (4/4)
SEXT.B	100.00% (10/10)
PACK	100.00% (3/3)
GREV	100.00% (2/2)
CSR Source	100.00% (3/3)
CSR Data	100.00% (9/9)
Error Reason	100.00% (5/5)
Error Valid	100.00% (2/2)

All 19 primary functional covergroups listed below reached 100% in the reported regression. This establishes stimulus closure for the approved coverage model; it does not imply DUT correctness. The scoreboard remains the authority for expected result and error checking, and known DUT mismatches remain visible even when the corresponding bins are covered.

Code Coverage Results

Metric	Measured Result	Interpretation
Block Coverage	100.00% (14/14)	Official Xcelium metric for instrumented/elaborated block points.
Expression Coverage	100.00% (3/3)	Official Xcelium metric; the denominator contains only instrumented expression conditions.
Toggle Coverage	58.67% (1045/1781)	Remaining points require reachability/scope classification; not all non-toggling bits are actionable in the standalone configuration.

Block and Expression coverage percentages do not demonstrate complete source-level RTL exercise by themselves. Xcelium reports only the procedural block and expression points instrumented after elaboration for the selected DUT scope. Therefore, 14/14 and 3/3 are valid official tool results for those points, but they are not counts of every begin/end block or every expression written in the RTL source.

Toggle coverage is lower because it requires each instrumented bit to change value. The fixed standalone configuration, constant or unused fields, disabled feature paths, and operand bits not toggled by this regression can leave toggle points uncovered even when every functional bin is hit. The reported 1045/1781 points are therefore reviewed individually; no manual source-level denominator is added to the official Xcelium results.

Verification Results

The recorded regression exercised all 21 approved BMU and CSR functional modes and closed the 19 primary functional-coverage groups. The specification-based reference model and cycle-accurate scoreboard evaluated correctness independently. The focused smoke test provides the legal NOP evidence; the consolidated regression provides the coverage and mismatch totals reported below.

The verification environment demonstrated defect-detection capability by reproducing specification-to-RTL mismatches in CTZ, CPOP, PACK, CSR write-source selection, GREV/REV8 byte ordering, control-legality/error handling, and SLT/MAX qualification. These findings are documented in the Confirmed DUT Findings table above and remain reproducible through focused operation tests.

Recorded scoreboard summary: 845 monitored samples, 1,689 checks, 1,467 passes, and 222 failures. The failures are split into 205 result mismatches and 17 error mismatches and are associated with the confirmed DUT findings; no UVM_FATAL setup failure was reported.

- Stimulus and functional coverage closure: achieved for the approved functional coverage model.
- Checker operation: active through the independent reference model and scoreboard.
- DUT correctness: not fully passing because confirmed design defects remain.
- Code coverage: reviewed separately from functional correctness and interpreted within the elaborated standalone configuration.
- Regression failures: accepted only when understood and linked to a confirmed DUT finding or documented limitation.

Risk Management

A risk-based verification approach is used to prioritize scenarios according to their potential impact on functional correctness, transaction alignment, and verification confidence. High-risk areas are mandatory in the final regression, medium-risk areas receive focused boundary/corner testing, and standard-risk areas are covered with representative directed stimulus plus limited randomization where it adds value.

Verification Area	Risk	Primary Risk	Mitigation / Verification Strategy
Reset behavior	High	Incorrect priority, stale result, or reset/valid interaction	Startup and active-traffic reset; valid_in=0/1; scoreboard checking.
One-cycle latency	High	Result associated with the wrong request	Cycle-aligned pending prediction; consecutive requests; final-response drain.
valid_in / result hold	High	Unexpected result update while no request is accepted	Directed hold intervals and 0->1 / 1->0 transitions.
Error handling / AP legality	High	Illegal controls accepted or error behavior misclassified	Dedicated negative sequence; conflict cases; error x valid_in coverage.
Testbench pipeline alignment	High	False failures caused by checker/driver timing	Smoke bring-up, controlled sequencing, monitor/scoreboard alignment checks.
Shift / rotate / BINV	Medium	Boundary amount/index handling	Boundary values and representative operand patterns.
Signed / unsigned compare	Medium	Incorrect signed interpretation	<, =, > relations, sign boundaries, mixed-sign operands.
CTZ / CPOP	Medium	Incorrect counts at operand boundaries	Zero, one-hot, all-one, count classes and byte-region stimulus.
GREV / REV8	Medium	Mode legality or byte permutation error	Legal REV8, unsupported modes, asymmetric byte patterns.
CSR data selection	Medium	Incorrect source selection / conflicts	Independent read/write-source tests and CSR/BMU conflicts.
Logical operations	Standard	Local datapath errors	Truth-pattern corners and representative operands.
SUB / SH2ADD	Standard	Borrow/wrap/qualifier defects	Arithmetic boundaries, wraparound, and qualifier cases.

Coverage holes are not automatically treated as verification failures. Each uncovered point is first classified as reachable, unreachable, configuration-disabled, integration-only, redundant, or outside the approved scope. Known DUT failures are preserved as reproducible tests rather than being masked to improve regression statistics.

Sign-Off Criteria

- Every in-scope operation/CSR mode has a mapped sequence/test and is observed in functional coverage.
- Expected results are checked through the independent reference model and scoreboard, including one-cycle result latency, reset, valid_in hold behavior, CSR selection, and error behavior.
- All approved and reachable functional coverage intents are closed or explicitly justified; low-value ignored-input coverage is not used as a sign-off gate.
- Xcelium code-coverage holes are reviewed individually. Block/Expression percentages are interpreted only within the elaborated/instrumented DUT scope, and Toggle gaps are classified.
- Every UVM error/failure is classified as a DUT defect, testbench issue, setup issue, or unresolved requirement; there are no unexplained UVM_FATAL/setup failures.
- Known DUT defects are documented with reproducible focused tests, expected versus actual behavior, and supporting logs/waveforms where appropriate.
- The final repository compiles from sim/filelist.f, supports the documented Makefile commands, and the full bmu_regression_test run produces reproducible logs and coverage reports.

Open Items, Assumptions and Clarification Requests

The following items remain explicit so they are not mistaken for verified requirements or silently excluded from closure:

- *CSR control combinations outside the three documented modes remain unresolved:* `csr_ren_in` asserted together with `csr_write`, and `csr_imm` asserted without `csr_write`. These combinations are excluded from pass/fail and functional sign-off until their priority and error behavior are clarified.
- *The 736 uncovered toggle points* must be classified from the detailed Xcelium report as reachable, constant, configuration-disabled, integration-only, or outside the approved standalone scope. Functional coverage at 100% does not close this review automatically.
- *The confirmed DUT defects* in CTZ, CPOP, PACK, CSR write-source selection, GREV/REV8, control legality, and SLT/MAX qualification remain open design findings. Their failing focused tests are retained as reproducible evidence rather than waived to make the regression pass.
- Final submission evidence should identify the regression seed, tool version, logs, coverage reports, and representative waveforms used for each confirmed finding.